B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

FPL Intelligence Platform

By
Johnson Mununkum

April 26, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.1 Objectives . . . 3
1.2 Chapter Overview . . . 5

2 Methodology **7**
2.1 Development Approach . . . 7
2.2 Requirements Determination . . . 8
2.3 Supervisor Meetings . . . 9
2.4 Technology Selection . . . 9
2.5 Testing and Validation . . . 10
2.6 Empirical Problem Solving . . . 11

3 Technology Review **12**
3.1 Python . . . 12
3.2 REST APIs and the FPL API . . . 13
3.3 SQLite . . . 13
3.4 Pandas . . . 14
3.5 Machine Learning and Supervised Learning . . . 15
3.6 Random Forest . . . 15
3.7 Feature Engineering . . . 17
3.8 XGBoost . . . 18
3.9 Linear Programming and PuLP . . . 19
3.10 Flask . . . 19
3.11 Bootstrap 5 . . . 20
3.12 GitHub and GitHub Actions . . . 20
3.13 Render . . . 21

4 System Design **22**
4.1 System Architecture Overview . . . 22
4.2 Data Collection and Processing Pipeline . . . 23
4.2.1 FPL API Integration . . . 24

4.2.2 Feature Engineering . . . 24
4.3 Database Design . . . 24
4.4 Machine Learning Pipeline . . . 26
4.4.1 Model Selection . . . 27
4.4.2 Training Process . . . 27
4.4.3 Model Evaluation . . . 27
4.4.4 Prediction Generation . . . 28
4.5 Squad Optimisation Module . . . 28
4.5.1 Problem Formulation . . . 28
4.5.2 Constraints . . . 28
4.5.3 Output . . . 29
4.6 Web Application Design . . . 29
4.6.1 Application Structure . . . 30
4.6.2 Frontend Design . . . 30
4.7 Deployment Architecture . . . 31
4.7.1 Render Web Service . . . 32
4.7.2 GitHub Actions Workflow . . . 32

5 System Evaluation 33
5.1 Objective 1: Data Collection and Feature Engineering . . . 33
5.2 Objective 2: Machine Learning Model . . . 34
5.2.1 Model Performance . . . 34
5.2.2 Prediction Consistency . . . 34
5.3 Objective 3: Squad Optimisation . . . 35
5.4 Objective 4: Web Application Deployment . . . 35
5.5 Objective 5: User Interface . . . 36
5.6 Limitations . . . 36

6 Conclusion 38
6.1 Project Summary . . . 38
6.2 Outcomes . . . 38
6.3 Reflections on Development . . . 39
6.4 Future Work . . . 40
6.5 Summary . . . 41

A 42
A.1 Source Code . . . 42
A.2 Live Application . . . 42
A.3 Installation Instructions . . . 42

List of Figures

4.1 FPL Intelligence Platform system architecture 23
4.2 FPL Intelligence Platform data pipeline 25
4.3 Features table schema 27
4.4 FPL Intelligence Platform home page 29
4.5 FPL Intelligence Platform predictions page 30
4.6 FPL Intelligence Platform squads page 31
4.7 FPL Intelligence Platform player profile page 31

List of Tables

4.1 FPL Intelligence Platform database schema 26

5.1 Machine learning model comparison 34
5.2 Squad optimisation constraint validation 35

Chapter 1

Introduction

Fantasy Premier League (FPL) is the official fantasy football game for the English Premier League, attracting over 10 million players worldwide each season [1]. Each participant manages a virtual squad of 15 Premier League players, earning points based on their actual match performances across a 38-gameweek season. Points are earned for goals scored, assists, clean sheets, saves, bonus points, and other statistical events, while deductions are applied for yellow cards, red cards, and goals conceded. Managers must make weekly decisions regarding player selection, captaincy, transfers, and squad composition, all within a fixed budget of £100 million and strict constraints requiring exactly 2 goalkeepers, 5 defenders, 5 midfielders, and 3 forwards.

The scale and complexity of FPL decision-making is significant. With around 750 registered players across 20 Premier League clubs, each with their own form, fixture schedule, injury status, and price, the number of possible squad combinations is very large. Each gameweek, a manager must also choose a captain whose points are doubled, choose a vice-captain, and select 11 players from their 15-man squad to start. The game also allows one free transfer per gameweek, with additional transfers costing four points each. These layered constraints make it difficult for any individual to manually evaluate all available options and consistently make optimal decisions.

Despite this complexity, FPL decision-making is largely driven by intuition, manual statistics analysis, community opinion, and social media discussion. While some third-party websites and tools exist to help managers, they are often limited to displaying raw statistics rather than providing data-driven predictions and recommendations. The volume of data generated each gameweek - covering player form, fixture difficulty, minutes played, clean sheets, goals, assists and more - makes it difficult for individual managers to consistently process all available information without the assistance of automated tools.

The English Premier League's official API provides free and comprehensive

access to all player statistics, fixture data, and gameweek results, making FPL an ideal domain for applying machine learning. The structured and regularly updated nature of this data means that supervised learning models can be trained on historical gameweek records and used to generate forward-looking predictions. Additionally, the well-defined constraints of squad selection - fixed budget, position limits, and club restrictions - make it a natural fit for mathematical optimisation techniques such as Linear Programming [2].

Machine learning has become widely used in sports analytics in recent years. Researchers have used it to predict player performance in football, basketball and cricket, with methods like Random Forests performing particularly well on sports statistics data [3]. Combining machine learning predictions with an automated team selection algorithm provides a smarter, data-driven way to help the millions of FPL managers who face these decisions every week.

The main problem this project solves is that there is no easy to use, data-driven tool available that combines player point predictions with squad selection for FPL managers. Most existing FPL prediction tools are either behind a paywall, incomplete, or only exist as Jupyter notebooks that require coding knowledge to run, making them inaccessible to the average manager. This project solves that by building a fully deployed, publicly accessible web application that automatically collects FPL data, predicts how many points each player will score in the next gameweek using a Random Forest model, and uses those predictions to generate optimal 15-player squads using Linear Programming. The system updates automatically every 6 hours throughout the season, keeping predictions current as player form and fixtures change.

This project is well suited for a final year Honours level project because it combines several technically challenging areas: collecting data from a REST API, engineering features, training a machine learning model, building an optimisation algorithm, developing a web application, and deploying it to the cloud. Each of the areas required independent research and design decisions, together demonstrating a wide range of software engineering and data science skills. The system is intended as a decision-support tool, with all final squad and transfer decisions left to the user.

1.1 Objectives

The main goal of this project is to design, build, and deploy an AI-powered FPL Intelligence Platform that predicts player points and generates optimal squad recommendations. The project was completed across two semesters. In the first semester, the focus was on building the data collection pipeline, engineering features, and training an initial machine learning model. In the second semester, the

squad optimisation algorithm was built, the full web application was developed and deployed to the cloud, and the system was tested and evaluated against the objectives set out below.

The project has the following specific objectives:

1. **Data Collection and Feature Engineering** Collect and store player statistics from the official FPL REST API, building a set of 10 features for the machine learning model. These features include rolling average points, minutes played, player form, opponent difficulty, home or away fixture [4], clean sheet rolling average, price, position, goals scored, and assists. The data pipeline must run automatically on a schedule to keep the database up to date throughout the season.

2. **Machine Learning Model** Train a machine learning model to predict how many points a player is likely to score in the upcoming gameweek. The target is an R^2 score greater than 0.45 on a separate test set, which would show the model is picking up real patterns in the data rather than just guessing. Mean Absolute Error (MAE) is also used to measure how accurate the predictions are in actual point terms. The model must produce consistent predictions across all pages of the web application.

3. **Squad Optimisation** Build a Linear Programming optimisation algorithm using the PuLP library to generate the best possible 15-player squads within all FPL rules, including a £100 million budget, position requirements (2 GK, 5 DEF, 5 MID, 3 FWD), and a maximum of 3 players from any single club. The system must produce 3 different squad options per request, with a captain recommendation in each squad based on the player with the highest predicted points.

4. **Web Application Deployment** Deploy the system as a live Flask web application on the Render cloud platform, accessible with a public URL. Data must update automatically every 6 hours using GitHub Actions to keep predictions current throughout the season. The application must remain available and responsive throughout the evaluation period.

5. **User Interface** Build an easy to use web interface where users can view predictions for all players, search and filter by position, explore individual player profiles with season statistics, and view the generated optimal squads with captain recommendations. The interface must support both light and dark modes and work properly on both mobile and desktop devices.

Success will be measured by the model's R^2 score and MAE on a separate test set, whether the generated squads correctly follow all the FPL rules and constraints, whether predictions are consistent across all pages of the application, and

whether the deployed web application remains available and responsive throughout the evaluation period.

1.2 Chapter Overview

The source code for this project is available on GitHub at `https://github.com/JohnsonMununkum/FantasyPremierLeaguewithML` and the live application can be accessed at `https://fpl-intelligence-web.onrender.com`. A screencast demonstration of the application is available at `https://youtu.be/8K4ZN0d-I4c`.

The repository is made up of the following main elements:

- `flask_app/app.py` - The main Flask application containing all API endpoints and HTML routes for the web interface.
- `flask_app/utils/data_fetcher.py` - The data collection and feature engineering pipeline, which gets player statistics and fixture data from the FPL REST API and saves the processed records to a SQLite database.
- `flask_app/utils/predictor.py` - The prediction module that loads the trained Random Forest model and generates point predictions for all players in the database.
- `flask_app/utils/optimizer.py` - The Linear Programming squad optimisation module, which selects the best 15-player squads within all FPL constraints using the PuLP library.
- `flask_app/utils/training.py` - The model retraining script, used to retrain the Random Forest model on the full season data at the end of the season.
- `flask_app/models/` - Contains the trained model file and the SQLite database used by the live application.
- `flask_app/templates/` - HTML templates for the home, predictions, player profile, and squad builder pages.
- `.github/workflows/update-fpl-data.yml` - The GitHub Actions workflow that automatically refreshes the data every 6 hours by calling the application's update endpoint on Render.

The remainder of this dissertation is organised as follows:

- **Chapter 2 - Methodology** describes the development approach used throughout the project, including the incremental and iterative process, the tools and platforms chosen, and how requirements were identified and validated across the two semesters of development.

- **Chapter 3 - Technology Review** covers the technologies, libraries, and algorithms used to build the system, including Python, Flask, scikit-learn, the Random Forest algorithm, PuLP Linear Programming, SQLite, Bootstrap 5, and the official FPL REST API. Each technology is explained at a conceptual level with references to relevant literature, showing that all the technology choices were well researched and suitable for the problem.

- **Chapter 4 - System Design** covers the overall system architecture, data flow, database structure, machine learning pipeline, optimisation algorithm, and user interface design, supported by diagrams and application screenshots. This chapter explains how the system was built - how each component was designed and how they all work together.

- **Chapter 5 - System Evaluation** measures the system against the objectives set out in this chapter. This includes testing the machine learning model using R^2 and MAE metrics, checking that the squad optimisation algorithm follows all FPL rules, functional testing of the deployed web application, and a discussion of the limitations and challenges encountered during development.

- **Chapter 6 - Conclusion** summarises the outcome of the project, reflects on the development process and what was learned, identifies the limitations of the current system, and suggests areas for future improvement and investigation.

Chapter 2

Methodology

This chapter describes how the FPL Intelligence Platform was developed. It covers the development approach used, how the project requirements were identified and tracked, how technologies were chosen, how supervisor meetings were run and how the system was tested and validated throughout the project.

2.1 Development Approach

This project used a software development methodology rather than a research-based one. Before any coding began, around six weeks were spent researching machine learning concepts and identifying which models would be best suited for predicting FPL player points. This research phase shaped the key design decisions made throughout the project, mainly around the choice of machine learning algorithms. After that, the main focus was on building a working, deployed software system [5].

Within the software development approach an incremental and iterative development model was used. Rather than trying to build the entire system at once, the project was split into phases, each producing a working component that could be tested and evaluated before moving on to the next. This allowed for continuous improvement based on feedback from my supervisor meetings and testing results, and made sure that progress was visible and measurable at each stage. This approach works particularly well for machine learning projects where model performance can only be measured once a working pipeline is in place and where improvements are made step by step based on evaluation results [5].

The project was structured across two semesters following the schedule in the project proposal. In Semester 1, the research and analysis phase ran from mid-October to mid-November, covering the FPL API, Linear Programming for team optimisation and machine learning models including Random Forest, XGBoost

and Neural Networks. The data collection and feature engineering phase followed from mid-November to late November with the Semester 1 review in January. In Semester 2, the optimisation system was built in late January, the front-end application was developed from Mid-February to the end of February, final integration and testing was done from the end of February to mid March, and the dissertation was written from March to early May.

A key advantage of the incremental approach was that it allowed the scope and architecture of the project to be adjusted based on the feedback. When the Semester 1 review board recommended that the project should produce a deployed application rather than a local Jupyter notebook, the Semester 2 work plan was updated to reflect this. This flexibility is one of the main strengths of an iterative development model, as it can accommodate changing requirements without needing the entire project to be redesigned from scratch.

2.2 Requirements Determination

The project requirements were identified at the start of Semester 1 through a project proposal. The proposal outlined the core objectives of the system, the technologies to be used, and the schedule of work for both semesters. This document was used as the main reference point throughout the project, allowing progress to be tracked against the original plan.

The initial requirements set out in the proposal were ambitious. The original objectives included collecting data from multiple sources such as the official FPL API, Understat, the Premier League website, and Fantasy Football Scout, training and comparing multiple machine learning models including Random Forest, XGBoost and Neural Networks, achieving an MAE of less than 2.5 points and a 70% top-10 player identification rate when tested on the 2023/2024 season, building price change prediction and captain selection optimisation, and carrying out user satisfaction testing with five or more FPL players.

During Semester 1, the initial prototype was built entirely using Jupyter notebooks. The data collection pipeline, feature engineering and machine learning models were all built and tested locally, with predictions printed to the notebook output rather than shown in a web interface. This was okay for the research and prototyping phase as the focus was on getting the data pipeline and model working rather than on how it looked. Following the Semester 1 review the board's feedback described above, the Jupyter notebook code was restructured into Python modules and deployed to the Render cloud platform as a Flask web application with automated updates through GitHub Actions.

As the project went on, the requirements were refined based on practical experience and supervisor feedback. Data collection was simplified to use only the

official FPL REST API, as it provided comprehensive and reliable player and fixture data without the added complexity of scraping multiple third-party websites. The model comparison was narrowed down to Random Forest and XGBoost, as Neural Networks were found to need a lot more training data than was available. Price change prediction and multi-season back-testing were removed from scope in order to deliver a fully deployed, publicly accessible application with automated updates. Random Forest was selected as the final model as it produced the best results, achieving an R^2 score of 0.503 and a MAE of 0.73, compared to XGBoost's R^2 score of 0.203 and MAE of 1.81.

Non-functional requirements included the need for the application to update automatically, be available throughout the season and provide a consistent user experience across devices. Extra requirements that came up during development included dark mode support, a player search and position filter and individual player profile pages.

2.3 Supervisor Meetings

Weekly meetings were held with my supervisor throughout both semesters. Each meeting we reviewed the progress made since the last meeting, discussed any issues faced and decided what tasks needed to be completed for the next meeting.

Notes were kept after each meeting using Notepad, recording the tasks agreed upon and any feedback or guidance given by my supervisor. The project proposal was used with these notes to track overall progress against the planned timeline and objectives, showing what had been done and what still needed to be done.

The weekly meetings provided regular structured feedback that helped keep the project on track and allowed issues to be spotted and fixed quickly. Having a weekly meeting also helped maintain consistent progress throughout the project, as each meeting some sort of progress needed to be shown.

Key decisions made during supervisor meetings included the move from Jupyter notebooks to a deployed Flask application, guidance on feature engineering and feedback on the dissertation chapters. The supervisor's input played a big role in shaping the final direction of the project, particularly the decision to focus on delivering a fully deployed and publicly accessible application over a more research-focused output.

2.4 Technology Selection

The original technology stack in the project proposal included Node.js and Express for the backend REST API, React with Tailwind CSS for the frontend, and Post-

greSQL as the relational database. During development, this stack was changed to a simpler, more unified architecture. Flask was chosen over Node.js because it allowed the entire backend including the machine learning pipeline, optimisation algorithm and REST API to be written in a single Python codebase, removing the need to manage two separate server environments. SQLite was chosen over PostgreSQL because its serverless, file-based architecture made deployment on Render simpler without needing a separately hosted database service. Bootstrap 5 was used instead of React and Tailwind CSS because the frontend requirements were mainly presentational rather than interactive and Bootstrap provided a responsive, professional interface with much less development overhead.

Python was chosen as the primary programming language due to its widespread use in data science and machine learning, its large library ecosystem and how easily it integrates with web frameworks [6]. The scikit-learn library was chosen for machine learning due to its well documented Random Forest implementation and built-in support for model evaluation metrics [7]. Flask was chosen as the web framework due to its lightweight design and suitability for building REST APIs in Python [8]. SQLite was chosen as the database due to its serverless architecture which made deployment on Render easy [9]. PuLP was chosen for linear programming due to its simple Python API for defining and solving optimisation problems [10].

XGBoost was tested as an alternative to Random Forest for the machine learning model, but performed worse on the FPL dataset ($R^2 = 0.203$, MAE = 1.81) compared to Random Forest's ($R^2 = 0.503$, MAE = 0.73) so XGBoost was not used in the final system. Neural Networks were also looked at but needed a lot more training data than was available from a single season to work properly.

GitHub was used throughout the project to track changes, with over 90 commits showing the progression from the data collection scripts to the fully deployed web application. GitHub Actions was used to automate data updates every 6 hours by calling the application's update endpoint on Render.

2.5 Testing and Validation

Testing was carried out throughout the development process at multiple levels. Each component of the system was tested on its own before being added to the rest of the application.

The machine learning model was evaluated using a train-test split, with 80% of the available gameweek data used for training and 20% for testing. The model's performance was measured using R^2 and Mean Absolute Error (MAE) on the test set. Multiple model configurations were tested including a number of estimators, maximum tree depth and minimum samples per leaf with the best performing one

selected for deployment. The final model achieved an R^2 score of 0.503 and a MAE of 0.73, which was greater than the target R^2 of 0.45 set in the objectives.

The squad optimisation algorithm was tested by checking that all the generated squads followed the FPL rules, total cost within budget, the correct number of players in each position and max of 3 players from each team. Test cases were run across a range of player datasets to confirm that the algorithm generated valid squads.

The web application was tested manually by going through all pages and checking that predictions, player profiles, and squad recommendations displayed properly and consistently. The consistency of predictions between the predictions page, player profile pages and squad builder page was tested and verified following fixes made during development. The deployed application on Render was also checked to confirm that automated data updates were running properly every 6 hours using GitHub Actions.

2.6 Empirical Problem Solving

An empirical approach was taken to solve the key technical challenges faced during the project. When problems came up, the approach was to first find the root cause by looking at the data and system behavior, research the actual issue, implement a fix, and then test the result against the expected behavior.

For example, when the initial machine learning model generated poor predictions due to the opponent difficulty feature being on the wrong scale, the problem was found by looking at the feature values in the database, researched by checking the FPL API documentation, and fixed by adding a rescaling function that moved the raw 1-5 fixture Difficulty Rating (FDR) values to the 1-10 scale used during training. Also, when prediction values were found to be inconsistent between the predictions page and the player profiles pages, the root cause was found at the player profile endpoint generating predictions on a single-row DataFrame instead of the full dataset, which caused the Random Forest model to produce different results. This was fixed by updating the endpoint to generate predictions on the full dataset and then filtering down to the requested player.

These examples show the empirical, evidence based approach to problem solving used throughout the project. Each problem was treated as a chance to better understand the system and the technologies used and all solutions were tested before being considered complete.

Chapter 3

Technology Review

This chapter reviews the key technologies, libraries and algorithms used in the FPL Intelligence Platform. Each technology is explained at a conceptual level with references to relevant literature and industry standards. The technologies covered are connected to the objectives set out in the introduction - data collection, machine learning, squad optimisation, web application development and automated deployment.

3.1 Python

Python is a high-level, general-purpose programming language first released by Guido van Rossum in 1991. It was designed with a focus on code readability and simplicity, following the principle that there should be one clear way to do something [6]. Python has become the most widely used language in data science and machine learning, driven by its large ecosystem of open-source libraries, its easy-to-read syntax and how easily it connects with web frameworks, databases and external APIs.

Python uses an interpreted execution model, meaning code is run line by line at runtime rather than being compiled to machine code. This makes Python well suited to data analysis and rapid prototyping as code can be written and tested step by step without a separate compilation step. The language supports multiple programming styles including procedural, object-oriented and functional programming giving developers choice in how they structure their code [6].

The widespread use of Python in scientific computing is largely down to its library ecosystem. Key libraries used in this project include NumPy for numerical computation, Pandas for data manipulation, scikit-learn for machine learning, and Requests for HTTP communication with external APIs. Python's package manager, pip makes it easy to install and manage these libraries and a requirements

file can be used to list all project dependencies for consistent deployment.

Python was chosen for this project because it allowed the entire system - data collection, feature engineering, optimisation and web development to be built in a single language and codebase. This removed the complexity of managing multiple environments and made both development and deployment simpler.

3.2 REST APIs and the FPL API

A REST (Representational State Transfer) API is a software approach for building web services that communicate over HTTP. REST was defined by Roy Fielding in his PhD thesis in 2000 and has since become the most widely used standard for web APIs [11]. A RESTful API gives access to resources such as players, fixtures or gameweeks, as URLs and clients interact with these resources using standard HTTP methods: GET to get data, POST to create data, PUT to update data and DELETE to remove data.

REST APIs return data in structured formats, mainly in JSON (JavaScript Object Notation). JSON is a lightweight, easy to read data format that can be parsed in Python using the built-in `json` module or the Requests library [12]. The stateless nature of REST means that each request from a client to a server must have all the information needed to process it, with no session data stored on the server between requests.

The official Fantasy Premier League API is a publicly accessible REST API provided by the Premier League [13]. The main endpoints used in this project are `/api/bootstrap-static/`, which returns a full snapshot of all current season data including player statistics, team information and gameweek metadata. The `/api/element-summary/{id}/` endpoint is used to get the full gameweek-by-gameweek history for each player, which is used for feature engineering and model training.

The FPL API is open and free to use, making it accessible to all developers. Data is returned in JSON format and updated regularly throughout the season as gameweeks are completed. Using an officially maintained API means the data collected is accurate and reliable, removing the need for web scraping which is more unreliable and can break when a website's structure changes.

3.3 SQLite

SQLite is a lightweight, serverless, self-contained relational database management system. Unlike traditional database systems such as PostgreSQL or MySQL, SQLite does not run as a separate server process - the entire database is stored in

a single file on disk and the database engine runs directly inside the application [9]. This makes SQLite very easy to set up and deploy, as no separate database server needs to be configured or maintained with the application.

SQLite uses the SQL (Structured Query Language) standard for defining and querying data, making it compatible with standard database tools and libraries. In Python, SQLite can be accessed through the built-in `sqlite3` module and the pandas library has a `read_sql_query` function that allows SQL queries to be run and results returned as DataFrames.

SQLite puts data into tables with rows and columns. Relationships between tables are defined using foreign keys, allowing data to be structured and queried efficiently. In this project, the SQLite database stores three main tables: `players_raw` which stores the raw player statistics from the FPL API, `features` which stores the engineered features used for predictions, and `current_gameweek` which stores the current gameweek number.

SQLite is used by a large number of production applications and is described by its authors as the most widely deployed database engine in the world [9]. It is a good fit for this project because the application runs on a single instance on Render, there is no need for multiple processes writing to the database at the same time and the serverless architecture removes the need to set up and pay for a separate database.

3.4 Pandas

Pandas is an open-source Python library for data manipulation and analysis. It provides two main data structures: the Series, a one dimensional labelled array, and the DataFrame, a two-dimensional table with labelled rows and columns [14]. DataFrames are the core data structure used throughout this project for storing player statistics, engineered features, and prediction results.

Pandas provides a large set of functions for data cleaning, transformation and aggregation. Operations such as sorting, filtering, grouping, joining and reshaping data can be done in just a few lines of code. The library also supports reading and writing data in many formats including CSV, Excel and SQL databases, making it straightforward to load data from the FPL API and store it in the SQLite database.

A key feature of Pandas used in this project is the `groupby` and `transform` functions, which are used to calculate rolling averages of player statistics over a sliding window of recent gameweeks. Rolling averages are a common technique in time series feature engineering, gathering recent trends in player performance more accurately than season-long averages. The `rolling` function in Pandas calculates these averages efficiently over the grouped data, allowing the features to

be calculated for all players in the dataset.

3.5 Machine Learning and Supervised Learning

Machine learning is a subfield of artificial intelligence focused on building algorithms that learn patterns from data and make predictions or decisions without actually being programmed for each case [15]. Machine learning algorithms are split into three types: supervised learning where a model is trained on labelled input-output pairs, unsupervised where the model finds structure in unlabelled data, and reinforcement learning where an agent learns by interacting with an environment and receiving rewards or penalties.

This project uses supervised learning, mainly regression to predict how many FPL points a player will score in the upcoming gameweek. In a regression task the goal is to learn a relationship between a set of input features and a continuous numerical output, in this case, the predicted points value. The model is trained on historical gameweek records, where features describe the player's recent form and upcoming fixture, and the target variable is the actual points the player scored in that gameweek [16].

The quality of a supervised learning model is measured using evaluation metrics. This project uses two metrics: R^2 (a measure of how well the model's predictions match the actual values) and Mean Absolute Error (MAE). R^2 measures how much of the variance in the target variable is explained by the model, with a value of 1.0 indicating a perfect fit and 0.0 meaning the model performs no better than predicting the mean value. MAE measures the average difference between predicted and actual values in the original units of the target variable - in this case, FPL points [16].

3.6 Random Forest

Random Forest is a supervised machine learning algorithm based on the idea of ensemble learning. It was introduced by Leo Breiman in 2001 and has become one of the most widely used algorithms for both classification and regression tasks [3]. A Random Forest is made up of a large number of individual decision trees, each trained on a random subset of the training data and a random subset of the available features. Predictions are made by combining the results of all trees, for regression tasks the average of all tree predictions is used as the final output.

Ensemble learning is a technique where multiple models are combined to produce a better overall prediction than any single model could on its own [3]. The key idea is that individual models each make different errors, and by combining

their predictions these errors tend to cancel each other out. In Random Forest, each tree is trained on a different random sample of the training data, a process known as bagging. This means each tree sees a slightly different view of the data and learns slightly different patterns. When predictions are averaged across all trees, the final prediction is more stable and accurate than any individual tree. The random selection of features at each split also increases the variety between trees, making sure that different trees capture different patterns in the data.

A decision tree is a model that learns a series of binary splitting rules on the input features to predict the target variable. At each node in the tree, the algorithm chooses the feature and threshold that best splits the data using a measure such as mean squared error for regression. The tree keeps growing until a stopping condition is met, such as a maximum depth or a minimum number of samples per leaf. Individual decision trees usually overfit. They can memorise the training data and perform poorly on unseen data, but the Random Forest prevents this problem by averaging the results of many trees each trained on different subsets of the data [3].

The key hyperparameters of the Random Forest algorithm include the number of estimators (the number of trees), the maximum depth of each tree, the minimum number of samples needed to split a node and the number of features used at each split. In this project, the model was set up with 200 estimators, a maximum depth of 10, a minimum of 20 samples per split and a minimum of 10 samples per leaf. These values were chosen through experimentation to get the best predictive performance while avoiding overfitting.

Random Forest gives a feature importance score for each input feature, calculated as the average reduction in impurity across all trees that use that feature for splitting. In this project, the `minutes` feature had the highest importance at 68.5% reflecting the strong link between playing time and FPL points. The `rolling_avg_points` feature had an importance of 13.0%, followed by `price` at 8.2%.

The Pearson correlation results confirmed that `minutes` had the strongest relationship with FPL points at $r = 0.685$, followed by `rolling_avg_points` at $r = 0.419$, `clean_sheets_rolling_avg` at $r = 0.325$ and `price` at $r = 0.308$. The `opponent_difficulty` and `is_home` features had weaker correlations of $r = 0.055$ and $r = 0.030$, but were kept as they provided useful information about the upcoming fixtures that the other features do not capture.

One known limitation of the Random Forest algorithm is that its predictions are limited by the range of values seen in the training data. Because predictions are averages of tree outputs, the model cannot predict values higher than the maximum values seen during training. This means that very high scores such as a player scoring 20 or more points in a single gameweek, are likely to be underestimated

by the model.

3.7 Feature Engineering

Feature engineering is the process of converting raw data into a set of input features that are useful for a machine learning model. The quality of the features used for training has a big impact on model performance. A model trained on well-engineered features will usually perform better than one trained on raw data, even if the same algorithm is used [17].

- **rolling_avg_points** — The average FPL points scored by the player over the last 5 gameweeks. This captures recent form and is the second strongest predictor of future performance after minutes played.
- **minutes** — The number of minutes played by the player in the most recent gameweek. Players who play more minutes score more points on average and this feature has the highest importance in the trained model.
- **opponent_difficulty** — A rating of how difficult the player's upcoming fixture is, scaled from 1 (easiest) to 10 (hardest). This is based on the FPL Fixture Difficulty Rating (FDR), rescaled from its original 1-5 range to 1-10 to better distinguish between fixture difficulties.
- **is_home** — A binary feature showing whether the player's next fixture is at home (1) or away (0). Home advantage is well documented in football [4] and home players tend to score more FPL points on average.
- **price** — The player's current FPL price in millions of pounds. Price is used as an indicator of overall quality, as more expensive players tend to be stronger performers.
- **clean_sheets_rolling_avg** — The rolling average of clean sheets over the previous 5 gameweeks, used only for goalkeepers and defenders where clean sheets are a big part of FPL points.
- **pos_GK, pos_DEF, pos_MID, pos_FWD** — One-hot encoded position features showing whether the player is a goalkeeper, defender, midfielder, or forward. These allow the model to learn different scoring patterns for each position.

The features were chosen based on how FPL points are scored and what factors most affect player performance. Minutes played was used because a player

cannot score any points without playing, making it the most important indicator of whether a player will play in a given gameweek. Rolling average points was included to capture recent form, as a player who has been scoring well over the last five gameweeks is more likely to keep performing well. Opponent difficulty was used because playing against a weaker team increases the chances of scoring goals and keeping clean sheets. Home advantage was used as research has shown that home teams consistently perform better than away teams in football [4]. Price was used as an indicator of quality, since the FPL pricing system broadly reflects a player's ability and expected output. Clean sheet rolling average was used mainly for goalkeepers and defenders, as clean sheets are one of the main sources of points for these positions. Position encoding was used to allow the model to learn the different scoring patterns for each position, since forwards score mainly through goals and assists while defenders and goalkeepers score through clean sheets and saves.

Rolling averages are calculated using the Pandas `rolling` function with a window of 5 gameweeks and a minimum of 5 periods, making sure that only players with enough recent history have non-zero rolling average values. The shift operation is applied to avoid data leakage, which would mean using the current gameweek's performance to predict itself.

3.8 XGBoost

XGBoost (Extreme Gradient Boosting) is a gradient boosting algorithm that builds an ensemble of decision trees one by one, with each new tree trained to fix the errors of the last one [18]. Unlike Random Forest, which builds trees independently at the same time, gradient boosting builds trees one after another, with each tree learning from the mistakes of the current ensemble. XGBoost is widely seen as one of the most effective algorithms for structured tabular data and has won many machine learning competitions [18].

XGBoost was tested as an alternative to Random Forest for this project. However, it generated worse results on the FPL dataset, achieving an R^2 of 0.203 and a MAE of 1.81 compared to the Random Forest model's R^2 of 0.503 and MAE of 0.73. This is likely because XGBoost is more likely to overfit on smaller datasets and the available training data of about 18,990 records was not enough for the algorithm to perform well on unseen data. As a result Random Forest was used for the final system.

3.9 Linear Programming and PuLP

Linear Programming (LP) is a mathematical optimisation technique used to find the best outcome in a model where all requirements are defined as linear relationships. It was developed by George Dantzig in the 1940s and his simplex method remains the basis of most LP solvers used today [2]. A linear programme consists of an objective function to be maximised or minimised, a set of variables, and a set of constraints defined as linear inequalities or equalities.

In this project, the squad selection problem is set up as an Integer Linear Programme (ILP). A binary decision variable x_i is assigned to each player i, taking the value 1 if the player is selected in the squad and 0 if not. The objective function maximises the total predicted points of the selected squad:

$$\text{maximise} \sum_i p_i x_i \tag{3.1}$$

- Total squad cost: $\sum_i c_i x_i \leq 100$, where c_i is the player's price in millions.
- Squad size: $\sum_i x_i = 15$.
- Position rules: exactly 2 goalkeepers, 5 defenders, 5 midfielders, and 3 forwards must be selected.
- Club limit: no more than 3 players from any single Premier League club.

PuLP is an open-source Python library for building and solving linear programming problems [10]. It provides a simple API for defining variables, objective functions and constraints in Python code and works with a range of solvers. In this project, the CBC (COIN-OR Branch and Cut) solver is used, which comes included with PuLP by default and can solve the ILP problems generated by the squad optimisation algorithm properly.

3.10 Flask

Flask is a lightweight web framework for Python, released by Armin Ronacher in 2010 [8]. It is called a microframework because it provides the minimum set of tools needed to build a web application, URL routing, request handling and response generation without forcing a specific project structure or including components such as an ORM or form validation library that are not always needed. This makes Flask well suited to projects where simplicity is more important than the structure of a full-stack framework such as Django.

Flask follows the WSGI (Web server Gateway Interface) standard, which defines a common interface between Python web applications and web servers [19]. This allows Flask applications to be deployed on any WSGI compatible server, including Gunicorn which is used to run this project on Render.

In this project, Flask does two things: it provides a REST API with endpoints for getting predictions, player details and optimised squads and it handles the HTML templates for the web interface using Jinja2, Flask's built-in template system. The REST API endpoints return JSON responses, while the HTML routes render templates that load data from the API using JavaScript fetch calls.

3.11 Bootstrap 5

Bootstrap is an open-source frontend framework for building responsive web interfaces, developed by Twitter and released in 2011 [20]. It provides a grid system, a collection of ready-to-use UI components including navigation bars, cards, tables and buttons and a collection of helper classes for controlling layout, spacing, typography and colour. Bootstrap is one of the most popular frontend frameworks in web development, with a large community and extensive documentation.

Bootstrap 5, the version used in this project, gave many improvements over previous versions including the removal of the jQuery dependency, an updated grid system based on CSS Flexbox and new utility classes [20]. The responsive grid system splits the page into 12 columns and automatically adjusts the layout to different screen sizes, making sure the application works properly on both desktop and mobile devices.

In this project, Bootstrap 5 is loaded from a Content Delivery Network (CDN) removing the need to add the library files in the project repository. The application uses Bootstrap components including a responsive navigation bar, prediction tables, player profile cards and squad display cards.

3.12 GitHub and GitHub Actions

Git is a distributed version control system created by Linus Torvalds in 2005 [21]. It tracks changes to source code over time, allowing developers to keep full history of all changes, go back to previous versions when needed and work on different features at the same time using branches. GitHub is a web-based platform that stores Git repositories and adds extra collaboration and automation features.

GitHub Actions is a continuous integration and deployment (CI/CD) platform built into GitHub [22]. It allows automated workflows to be defined in YAML files stored in the repository. Workflows are triggered by events such as code pushes,

pull requests or scheduled timers and they run jobs on GitHub hosted virtual machines. In this project, the GitHub Actions workflow runs every 6 hours, sending an HTTP POST request to the application's `/api/trigger-update` endpoint on Render to trigger a data update. This keeps predictions current throughout the season without any manual work.

3.13 Render

Render is a cloud platform for deploying web applications, APIs and background workers [23]. It is known as a Platform as a Service (PaaS), meaning that the underlying infrastructure, virtual machines, networking and operating system configuration is managed by the platform, letting developers focus on their application code without having to manage servers directly.

In this project, the Flask application is deployed on Render as a web service. Render automatically detects the Python environment, installs dependencies from `requirements.txt` and starts the application using the Gunicorn WSGI server. The service is connected to the GitHub repository, so new deployments are triggered automatically when changes are pushed to the codebase. Render generates a public URL for the deployed application and handles HTTPS security certificates automatically.

Render's free tier was used for this project, which gives enough resources for a single server Flask application with scheduled automated updates. The main limitation of the free tier is that the service shuts down after some time of no activity and takes a short time to restart on the next request, which can cause some delay on the first visit to the application after some period of no use.

Chapter 4

System Design

This chapter describes the design and architecture of the FPL Intelligence Platform. It shows how the system was structured, how its components interact and the key design decisions made during development. The chapter is split into seven sections showing the overall system architecture, data collection pipeline, database design, machine learning pipeline, squad optimisation module, web application design and deployment architecture. Diagrams are used to demonstrate both the high-level structure of the system and the detail of individual components.

4.1 System Architecture Overview

The FPL Intelligence Platform is a web-based application made up of five main components that work together to collect player data, generate predictions, optimise squad selection and display results to the user. Figure 4.1 shows a high-level overview of the system architecture.

The system is made up of five components. The Data Collection Module connects to the official FPL REST API over HTTP to get player statistics, fixture data and gameweek results. The SQLite Database stores all player data and processed features and is updated automatically on a regular basis. The Machine Learning pipeline reads data from the database, trains a Random Forest model and generates point predictions for each player. The Squad Optimisation Module uses linear programming through the PuLP library to pick the best possible squad within the FPL budget and formation rules. The Flask Web Application brings everything together and shows predictions, squad recommendations and player profiles to the user using a Bootstrap 5 interface deployed on Render.

The components are loosely coupled by design. The data collection module runs independently on a schedule through GitHub Actions and writes to the database without depending on the web application. The machine learning pipeline reads

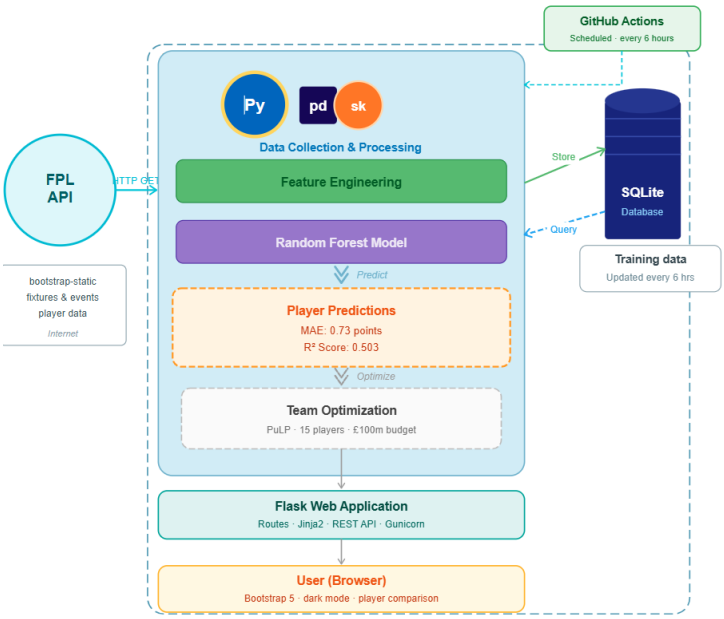

Figure 4.1: FPL Intelligence Platform system architecture

from the database and writes predictions back to it, also independent of the web layer. The Flask application reads from the database when a request comes in and returns the results to the user. This separation means each component can be updated without affecting the others and the data pipeline keeps running even when the web application is not being used.

The system is fully cloud hosted. The Flask application and SQLite database are deployed on Render as a PaaS web service, and the scheduled data updates run on GitHub Actions. This means no local infrastructure is needed and the application can be used from any device with a browser.

4.2 Data Collection and Processing Pipeline

The data collection and processing pipeline gets player data from the FPL API, engineers features from the raw data and stores the results in the SQLite database. It runs automatically every six hours through a GitHub Actions scheduled workflow, keeping the application up to date with the latest gameweek data.

4.2.1 FPL API Integration

The official FPL API is a public REST API that returns player statistics, fixture data and gameweek results in JSON format. The two main endpoints used are the `bootstrap-static` endpoint, which returns all the player data for the current season, and the `element-summary/id` endpoint, that returns the gameweek history for a specific player.

The data collection module sends HTTP GET requests to these endpoints using the Python `requests` library. The response is parsed as JSON and loaded into a Pandas DataFrame for processing. Error handling is added so that if a request fails, the pipeline logs the error and keeps the most recent successful data in the database rather than overwriting it with incomplete results.

4.2.2 Feature Engineering

Raw player data from the FPL API has cumulative season statistics such as total goals, total assists and total points. These figures alone are not suitable for predicting future gameweek performances as they do not capture recent form. Feature Engineering converts this raw data into features that better represent a player's current performance level.

The key features engineered for each player are as follows. Rolling averages are calculated over the last five gameweeks for total points, goals, assists, bonus points and minutes played. These capture form and give more weight to recent performances than season totals. Fixture difficulty is encoded as a numerical value from the FPL API's fixture difficulty rating, showing how hard the upcoming opponent is. Position encoding converts a player's position into four binary columns - `pos_GK`, `pos_DEF`, `pos_MID` and `pos_FWD` using one-hot encoding. A home or away indicator is also included as a binary feature showing whether the upcoming fixture is at home or away.

Once the features are engineered, the DataFrame is written to the `features` table in the SQLite database. A separate `next_gw_features` table stores only the features for the upcoming gameweek for each player, which is used at prediction time. This keeps the full historical data for training separate from the most recent data used to generate predictions.

4.3 Database Design

The FPL Intelligence Platform uses SQLite as its database, chosen for its serverless, file-based architecture, which simplified deployment on Render without needing a separately hosted database service.

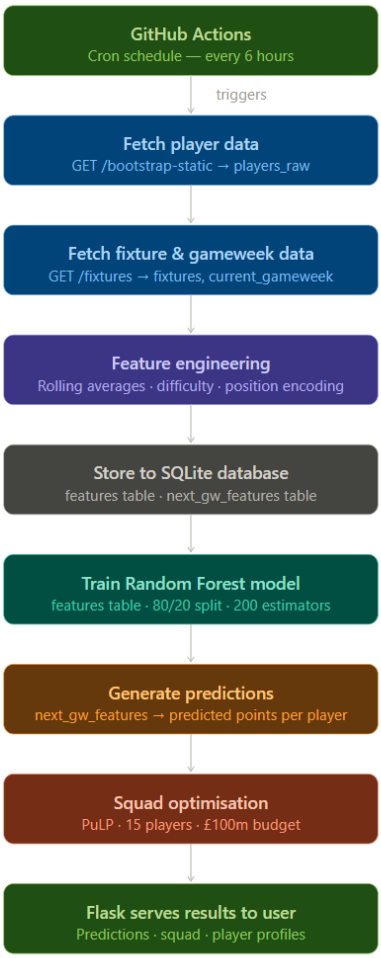

Figure 4.2: FPL Intelligence Platform data pipeline

The database is split across two SQLite files. The main application database has six tables. The `players_raw` table stores raw player data from the FPL API including goals, assists, bonus points and minutes played. The `features` table stores the engineered features used to train the Random Forest model. The `fixtures` table stores upcoming fixture data and difficulty rating from the FPL API. The `teams` table stores the Premier League team name, ID's, position and team strength. The `current_gameweek` table stores the current gameweek number to make sure features and predictions are always generated for the correct gameweeks. The `last_update` table stores the timestamp of the most recent data update.

A second database was used during earlier development and contains four ta-

bles. The `gameweek_data` table stores historical gameweek records used for model training. The `features` table stores the engineered features from that phase of development. The `next_gw_features` table stores the engineered features for the upcoming gameweek used at prediction time. The `players` table stores processed player data from that phase of development. This database was replaced by the main application database when the project moved from Jupyter Notebooks to the deployed Flask application.

Table	Column	Description
`current_gameweek`	current_gameweek	Current gameweek number
`features`	player_id name team gameweek position rolling_avg_points opponent_difficulty minutes is_home price pos_GK, pos_DEF, pos_MID, pos_FWD clean_sheets_rolling_avg	Player identifier Player name Player team Gameweek number Player position 5 gameweek rolling average Fixture difficulty rating Minutes played Home or away indicator Player price One-hot position encoding Rolling clean sheet average
`fixtures`	event, kickoff_time, team_h, team_a	Fixture data
`last_update`	last_update	Last update timestamp
`players_raw`	goals, assists, bonus, minutes, etc.	Raw FPL API data
`teams`	id, name, strength, etc.	Premier League team data

Table 4.1: FPL Intelligence Platform database schema

4.4 Machine Learning Pipeline

The machine learning pipeline trains the predictive model and generates points predictions for all players. It is implemented in `utils/predictor.py` and runs automatically at the end of each data update cycle.

Rows: 820 Filter 820 rows... Upgrade to PRO

	player_id	name	team	gameweek	position	rolling_a...	opponen...	minutes	is_home	price	pos
I	17	Martin Ødegaard	Arsenal	29	MID	1	10	36.89655172413793	1	7.8	
II	16	Bukayo Saka	Arsenal	29	MID	5.8	10	65.6896551724138	1	9.8	
1	1	David Raya Martín	Arsenal	29	GK	3.2	10	90	1	6	
2	2	Kepa Arrizabalaga Revuelta	Arsenal	29	GK	0	10	0	1	4.1	
3	3	Karl Hein	Arsenal	29	GK	0	10	0	1	4	
4	4	Tommy Setford	Arsenal	29	GK	0	10	0	1	3.9	
5	5	Gabriel dos Santos Magalhães	Arsenal	29	DEF	5.8	10	71.55172413793103	1	7.2	
	6	William Saliba	Arsenal	29	DEF	2.6	10	68.41379310344827	1	6.1	
7	7	Riccardo Calafiori	Arsenal	29	DEF	0.6	10	47.3448275862069	1	5.6	
8	8	Jurriën Timber	Arsenal	29	DEF	5.4	10	83.27586206896552	1	6.3	
9	9	Jakub Kiwior	Arsenal	29	DEF	0	10	0	1	5.4	
10	10	Myles Lewis-Skelly	Arsenal	29	DEF	0	10	10.724137931034482	1	5.1	
11	11	Benjamin White	Arsenal	29	DEF	0	10	14.10344827586207	1	5.1	
12	13	Brayden Clarke	Arsenal	29	DEF	0	10	0	1	3.8	
13	14	Maldini Kacurri	Arsenal	29	DEF	0	10	0	1	3.9	
14	15	Josh Nichols	Arsenal	29	DEF	0	10	0	1	3.7	
15	16	Bukayo Saka	Arsenal	29	MID	5.8	10	65.6896551724138	1	9.8	
16	17	Martin Ødegaard	Arsenal	29	MID	1	10	36.89655172413793	1	7.8	
17	18	Noni Madueke	Arsenal	29	MID	2.8	10	28.586206896551722	1	6.8	
18	19	Gabriel Martinelli Silva	Arsenal	29	MID	1.2	10	27.93103448275862	1	6.8	
19	20	Leandro Trossard	Arsenal	29	MID	1.4	10	57.758620689655174	1	6.7	
20	21	Declan Rice	Arsenal	29	MID	5.2	10	85.86206896551724	1	7.4	
821	22	Mikel Merino Zazón	Arsenal	29	MID	0	10	34.3448275862069	1	5.4	

Page 1 / 9 Copy as Excel

Figure 4.3: Features table schema

4.4.1 Model Selection

A Random Forest regressor was selected as the prediction model. Random Forest is an ensemble learning method that builds a large number of decision trees during training and outputs the average prediction across all trees [3]. It was chosen because it handles mixed feature types well, is resistant to overfitting through bagging and provides feature importance scores. The scikit-learn implementation was used with 200 estimators, a maximum depth of 10 and a fixed random state of 42 for reproducibility [7].

4.4.2 Training Process

The model is trained on all the data in the `features` table. The input features used are `position`, `minutes`, `rolling_avg_points`, `opponent_difficulty`, `is_home`, `price`, `pos_GK`, `pos_DEF`, `pos_MID`, `pos_FWD` and `clean_sheets_rolling_avg`. The target variable is `total_points`. The data is split 80/20 into a training set and a test set, and the model is evaluated using Mean Absolute Error and R^2 score.

4.4.3 Model Evaluation

The trained model achieved a MAE of 0.73 and an R^2 score of 0.503 on the test set. An MAE of 0.73 indicates that on average the model's predictions are less than one

point of the actual gameweek score, which is acceptable given the unpredictability of individual player performance in football. The R^2 score of 0.503 means the model explains around 50% of the variance in player points, which is a good result given that performance is also influenced by factors not captured in historical data such as injuries, managerial decisions and in-game events [24].

4.4.4 Prediction Generation

After training, the model is applied to `next_gw_features` table to generate the predicted points for each player. The predictions are written back to the database, replacing the old set. The trained model is also saved to the disk as a pickle file using the Python `joblib` library, so it can be loaded without retraining.

4.5 Squad Optimisation Module

The squad optimisation module selects the best possible FPL squad within the FPL constraints. It is implemented in `utils/optimizer.py` using the PuLP linear programming library [10].

4.5.1 Problem Formulation

The optimisation problem is formulated as an integer linear program. Each player has a binary decision variable $x_i \in \{0, 1\}$, where 1 means the player is selected and 0 means they are not. The objective is to maximise the total predicted points of the selected squad:

$$\text{Maximise} \quad \sum_{i=1}^{n} p_i x_i$$

where p_i is the predicted points for player i and n is the total number of players available

4.5.2 Constraints

The optimiser applies the following FPL constraints. The total cost of the selected squad must not be greater than £100 million. The squad must have exactly 15 players: 2 goalkeepers, 5 defenders, 5 midfielders and 3 forwards. No more than 3 players can be selected from the same club. Each constraint is implemented in PuLP and solved using the CBC solver.

4.5.3 Output

The solver returns the 15 players that maximise total predicted points within the given constraints. The result is passed to the Flask route and displayed in the squad optimiser view, showing each player's name, position, club, cost and predicted points.

4.6 Web Application Design

The web application shows the user interface for the FPL Intelligence Platform. It is built using Flask and accessed through a standard web browser. Flask was chosen because it allowed the machine learning pipeline, optimisation module and web layer to all be written in a single Python codebase [8].

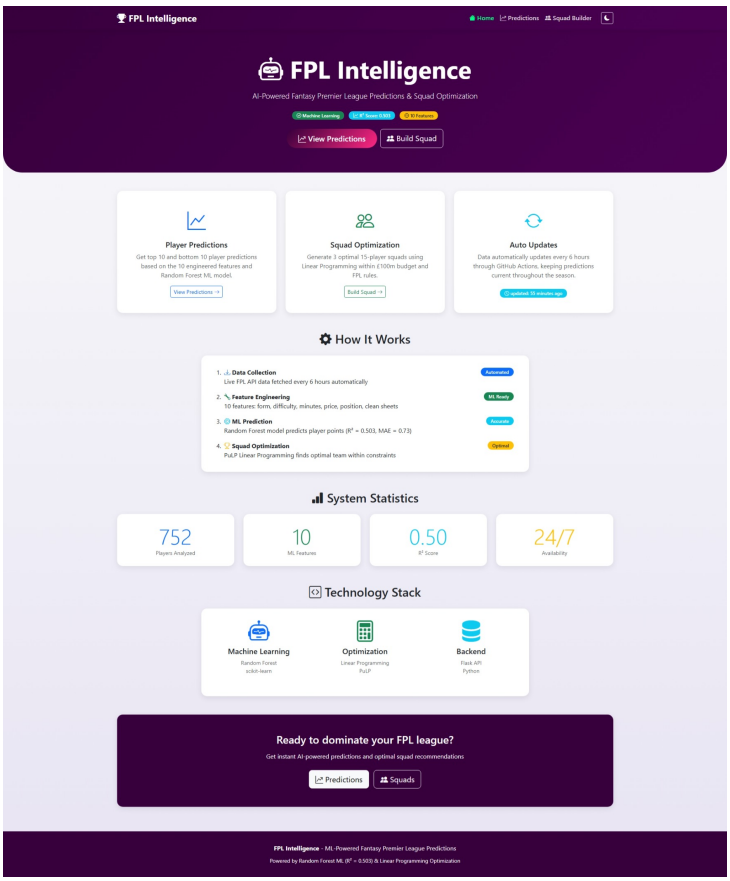

Figure 4.4: FPL Intelligence Platform home page

4.6.1 Application Structure

The application follows a standard Flask project structure. The entry point is `app.py`, which defines all routes and starts the application. Templates are rendered using Flask's built-in Jinja2 engine, which loads dynamic data from the database into HTML templates at request time. The main routes are: `/` for the home page, `/predictions` for the predictions dashboard, `/squads` for the squad builder and `/player/<id>` for individual player profiles. The main API routes are: `/api/predictions/top` and `/api/predictions/bottom` for top and bottom predicted players, `/api/optimize` for a single optimal squad, `/api/optimize/multiple` for three squad options and `/api/trigger-update` which is called by GitHub Actions every six hours to update the data.

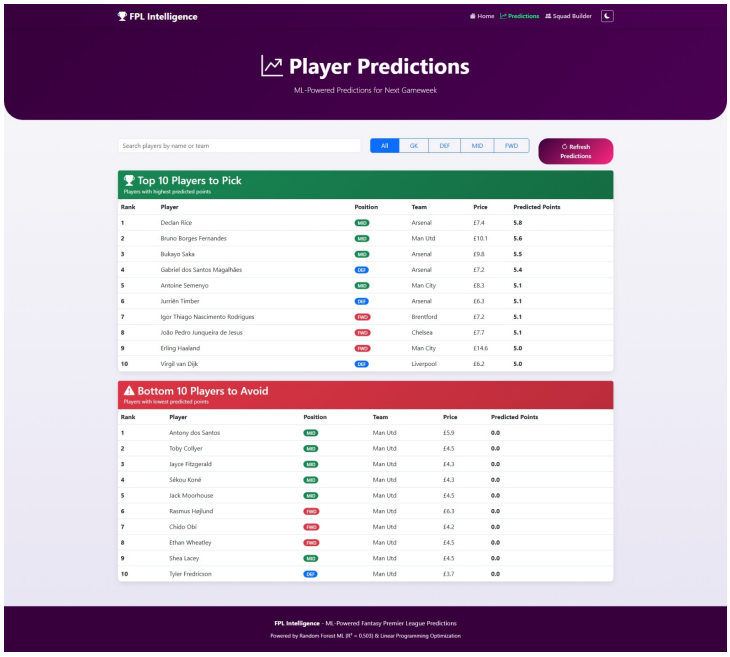

Figure 4.5: FPL Intelligence Platform predictions page

4.6.2 Frontend Design

The frontend is built using Bootstrap 5, which provides a responsive layout and pre-styled components without needing a JavaScript framework [20]. A dark mode toggle is implemented using a JavaScript class on the document body, with the user's preference saved in the browser's local storage so it is kept across sessions. Player search and filtering is handled client-side using JavaScript so the predictions table can be filtered by name, position or club without a server request.

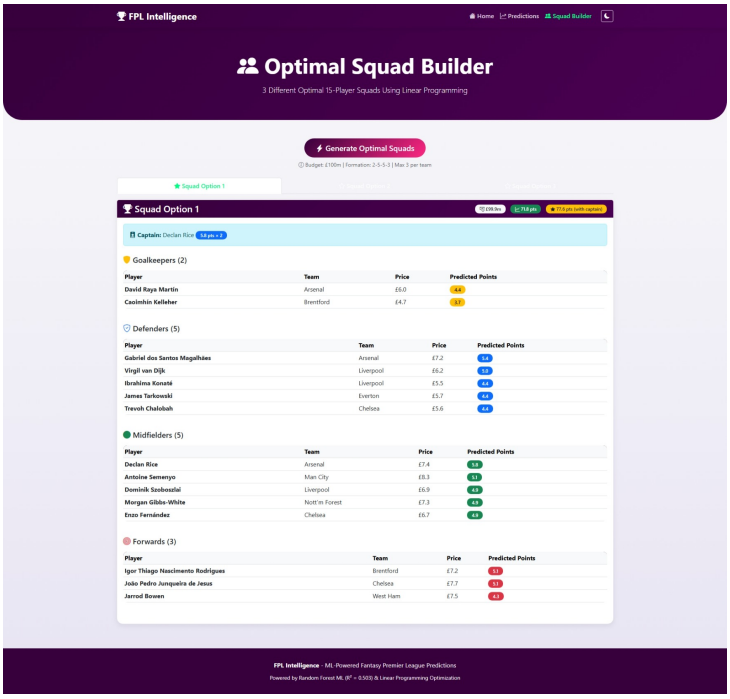

Figure 4.6: FPL Intelligence Platform squads page

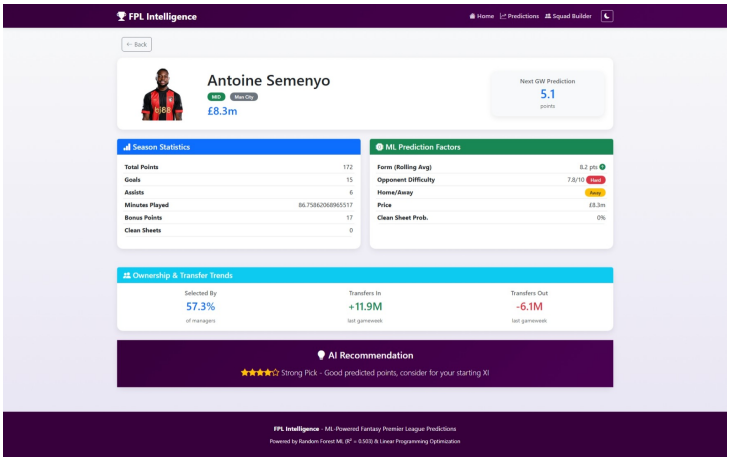

Figure 4.7: FPL Intelligence Platform player profile page

4.7 Deployment Architecture

The FPL Intelligence Platform is fully deployed using cloud services with no local infrastructure needed. The deployment is made up of two components: the

Render web service hosting the Flask application and the GitHub Action workflow handling scheduled data updates.

4.7.1 Render Web Service

The Flask application is deployed on Render as a web service. Render is a PaaS provider that manages the underlying infrastructure, allowing the application to be deployed by providing a runtime environment and start command [23]. Render detects the Python environment automatically, installs dependencies from `requirements.txt` and starts the application using the Gunicorn WSGI server. The service is connected to the GitHub repository so new deployments are triggered automatically when changes are pushed. The application is accessible at `https://fpl-intelligence-web.onrender.com`.

4.7.2 GitHub Actions Workflow

Scheduled data updates are handled by a GitHub Actions workflow defined in `.github/workflows/update-fpl-data.yml`. The workflow runs every six hours on a cron schedule and calls the `/api/trigger-update` endpoint on Render, which runs the data collection and feature engineering pipeline against the live FPL API. This removes the need for a separately hosted task scheduler and keeps the application up to date throughout the season.

Chapter 5

System Evaluation

This chapter evaluates the FPL Intelligence Platform against the five objectives set out in the introduction. Each objective is assessed using the success measures defined at the start of the project. This chapter ends with a discussion of the limitations of the system.

5.1 Objective 1: Data Collection and Feature Engineering

The first objective was to collect and store player statistics from the official FPL REST API and build a set of 10 features for the machine learning model. The data pipeline also needed to run automatically on a schedule to keep the database up to date throughout the season.

This objective was achieved. The data collection pipeline gets player statistics and fixture data from the FPL API using the `bootstrap-static` and `element-summary/id` endpoints. The raw data is stored in the `players_raw` and `fixtures` tables in the SQLite database. Feature engineering then converts this raw data into the 10 features used by the model: rolling average points, minutes played, opponent difficulty, home or away indicator, clean sheet rolling average, price and the four one-hot encoded position columns `pos_GK`, `pos_DEF`, `pos_MID` and `pos_FWD`. The engineered features are stored in the `features` table, with a separate `next_gw_features` table storing only the upcoming gameweek features used at prediction time.

The pipeline runs automatically every six hours through a GitHub Actions scheduled workflow, which calls the `/api/trigger-update` endpoint on Render. This keeps the database current throughout the season without any manual intervention. The `last_update` table records the timestamp of each update, confirming the pipeline is running as expected.

5.2 Objective 2: Machine Learning Model

The second objective was to train a machine learning model to predict player points for the upcoming gameweek, aiming for an R^2 score greater than 0.45 on unseen data. MAE was also used to measure how accurate the predictions were in actual point terms. The model was also needed to generate consistent predictions across all pages of the web application.

5.2.1 Model Performance

The Random Forest model was trained on all the data in the `features` table using an 80/20 train-test split. The final model achieved an R^2 of 0.503 and a MAE of 0.73 on the test set. The R^2 target of 0.45 was passed, showing that the model is picking up real patterns in the data. An MAE of 0.73 means that on average the model's predictions are less than one point of a player's actual gameweek score, which is a good result given how unpredictable individual player performance in football can be.

During development, an earlier version of the model achieved an R^2 of 0.315 and a MAE of 0.83. The improvement came from adding clean sheet rolling averages and price to the feature set, adjusting the hyperparameters to 200 estimators with a maximum depth of 10, and fixing a bug where the fixture difficulty feature was on the wrong scale. These changes helped the model better separate high-scoring and low-scoring players.

Model	$\mathbf{R^2}$	**MAE**
Random Forest (initial)	0.315	0.83
Random Forest (final)	0.503	0.73
XGBoost	0.203	1.81

Table 5.1: Machine learning model comparison

5.2.2 Prediction Consistency

During testing, predictions on the player profile pages were found to be different from those on the predictions page. This was because the player profile endpoint was generating predictions on a single-row DataFrame instead of the full dataset, which caused the Random Forest model to generate different results. This was fixed by generating predictions on the full dataset and then filtering down to the requested player. After this fix, predictions were consistent across the predictions page, squad builder and all player profile pages.

5.3 Objective 3: Squad Optimisation

The third objective was to build a Linear Programming optimisation algorithm using PuLP to generate the best possible 15-player squads with the FPL constraints. The system needed to generate 3 different squad options, with a captain recommendation based on the player with the highest predicted points.

This objective was achieved. The optimisation module in `utils/optimizer.py` selects the best squad by maximising total predicted points with the following constraints: a total cost equal to or under £100 million, exactly 2 goalkeepers, 5 defenders, 5 midfielders and 3 forwards, and a maximum of 3 players from the same club. The CBC solver with PuLP solves the problem and returns the optimal 15 players.

Constraint	**Requirement**	**Result**
Budget	£100 million	Passed
Goalkeepers	2	Passed
Defenders	5	Passed
Midfielders	5	Passed
Forwards	3	Passed
Players per club	3 maximum	Passed
Squad size	15	Passed

Table 5.2: Squad optimisation constraint validation

The algorithm was tested against a range of player datasets to show that all generated squads followed the FPL rules. Each generated squad was valid, with the budget, position counts and club limits all correct. The system generates 3 different squads through the `/api/optimize/multiple` endpoint, with the captain in each squad being the player with the highest predicted points.

5.4 Objective 4: Web Application Deployment

The fourth objective was to deploy the system as a live Flask web application on Render, accessible through a public URL, with data updating automatically every six hours through GitHub Actions. The application needed to stay available and responsive throughout the testing period.

This objective was achieved. The application is deployed at `https://fpl-intelligence-web.onrender.com` and is publicly accessible without any login or setup required. Render automatically detects the Python environment, installs dependencies from `requirements.txt` and starts the application using Gunicorn. New deployments are triggered automatically when changes are pushed to the GitHub repository.

The automated update pipeline ran throughout the testing period, with the `last_update` table showing updates every six hours. The application remained available with no downtime needed. The one limitation is that Render's free tier puts the service to sleep after a period of no use, causing a delay of 30 to 60 seconds on the first visit after the service has been inactive.

5.5 Objective 5: User Interface

The fifth objective was to build a web interface where users can view player predictions, search and filter by position, view individual player profiles and optimal squads with captain recommendations. The interface supports both light and dark modes and works on mobile and desktop devices.

This objective was achieved. The web application has four main pages. The home page shows an overview of the platform including the current gameweek, number of players tracked and last update time. The predictions page shows all players ranked by predicted points, with search and filter controls to filter by name, position or club. Individual player profiles are accessible at `/player/<id>` and show the player's season statistics and predicted points. The squad builder page shows three optimal squad options, each with captain recommendations.

Dark mode is implemented using a JavaScript class on the document body, with the preference saved in the browser's local storage so it is kept across sessions. Bootstrap 5 makes sure the layout works on both mobile and desktop. All pages were manually tested on desktop and mobile browsers during development.

5.6 Limitations

While all objectives were achieved, there are a number of limitations in the current system.

The machine learning model is trained on a single season of data. Player performance is influenced by many factors not captured in the statistics, such as injuries, rotation risk and managerial decisions. As a result, around 50% of the variance in player points is still unexplained. A model trained on multiple seasons would likely produce better predictions, but the FPL API only provides data for the current season.

The squad optimisation algorithm selects the best squad for the next gameweek only. It does not consider longer-term planning such as fixture difficulty over the upcoming weeks or transfer planning. A more advanced model would consider these factors to generate squads that perform well over a run of fixtures rather than just one gameweek.

The application is on Render's free tier, which causes a delay of 30 to 60 seconds when the service has been inactive. Upgrading to a paid tier would remove this issue.

The current interface only shows captain recommendations on the squad builder page and does not provide transfer suggestions for users with an existing squad. Adding these features would make the application more useful for active FPL managers.

Chapter 6

Conclusion

This chapter summarises the FPL Intelligence Platform project, reflects on what was built and what was learned during development, and looks at what could be improved or extended in the future.

6.1 Project Summary

Fantasy Premier League is played by over 10 million managers worldwide each season, yet most decisions are still based on intuition and community opinion rather than data. This project set out to build a fully deployed, publicly accessible web application that uses machine learning to predict how many points each FPL player will score in the upcoming gameweek and uses those predictions to generate optimal 15-player squads using Linear Programming. The system was built to update the data automatically throughout the season, so no manual data collection or model retraining is needed.

The project combined several technically challenging areas: collecting data from a public REST API, engineering features from raw player statistics, training a machine learning model, building a linear programming optimisation algorithm, developing a web application and deploying it to the cloud with automated updates. Each area required independent research and design decisions.

6.2 Outcomes

All five objectives set out in the introduction were achieved. The main outcomes are listed below:

- The data collection pipeline gets player statistics and fixture data from the FPL API every six hours through a GitHub Actions scheduled workflow,

keeping the database up to date throughout the season.

- A Random Forest model was trained on engineered features from the available gameweek data, achieving an R^2 of 0.503 and a MAE of 0.73, passing the target R^2 of 0.45 and generating predictions with less than one point of a player's actual gameweek score on average.
- A Linear Programming squad optimisation algorithm was built using PuLP, generating 3 different optimal 15-player squads and correctly applying all FPL constraints including budget, position counts and club limits in all generated squads.
- The system was deployed as a publicly accessible Flask web application at `https://fpl-intelligence-web.onrender.com`, with automated data updates every six hours through GitHub Actions and no downtime during the testing period.
- A responsive web interface was built using Bootstrap 5, with player predictions, individual player profiles, squad recommendations with captain suggestions, search and filtering, dark mode and mobile support.

6.3 Reflections on Development

The most valuable part of this project was building and deploying a full data science system rather than just a Jupyter Notebook. Early in the project, all the work was done locally in notebooks. The Semester 1 review board feedback pushed the project towards building a fully deployed application, which turned out to be the right decision. Restructuring the notebook code into Python modules, connecting them through a Flask application and deploying everything to Render was challenging but gave a much better understanding of how real data science systems are built and maintained.

Working with the FPL API [13] gave some unexpected findings. One finding was how much fixture difficulty affects predicted points. Early versions of the model did not correctly process the fixture difficulty rating and generated worse predictions, particularly for defenders and goalkeepers whose clean sheet chances depended heavily on the opponent. This showed how important good feature engineering is compared to using a more advanced model.

Another unexpected finding came from the prediction consistency bug. The finding that a Random Forest model can generate different predictions depending on whether a single row is given or the full dataset was not expected during the design phase. Finding and fixing this issue gave a much better understanding of

how ensemble models work and how small implementation details can have a big impact on results seen by users.

The decision to switch from Node.js, React and PostgreSQL to Flask, Bootstrap and SQLite was a good lesson in choosing the right technologies. The original stack was chosen because it was familiar, but combining Python machine learning code with a Node.js backend would have added a lot of complexity. Switching to a single Python codebase made everything simpler and faster to build, and showed the value of choosing technologies that fit the problem rather than using what is already known.

6.4 Future Work

While all five objectives were achieved, there are several clear directions for future development.

The most significant improvement would be training the model on multiple seasons of data. The current model is trained on a single season, which limits how well it performs across different seasons and player patterns. Training on multiple seasons would likely produce a more reliable model, particularly for spotting players who consistently score well across different opponents and situations.

The squad optimisation algorithm could be improved to support long-term planning. The current algorithm selects the best squad for a single gameweek. A more advanced version would consider fixture difficulty over the next five gameweeks, helping managers spot players with a good run of fixtures rather than just those predicted to score highly in one game. Transfer planning and chip usage such as when to use the Wildcard, Free Hit or Triple Captain could also be added to the optimisation model.

The user interface could be improved to include transfer recommendations for managers with an existing squad. Instead of generating a squad from scratch, the system could suggest which players to bring in and take out based on predicted points and budget, making it more useful for active FPL managers during the season.

Adding a player injury and availability tracker would improve the reliability of predictions. The current model does not know if a player is injured or suspended, which can lead to high predicted points for players who will not play. Adding availability data from the FPL API would allow the pipeline to filter out unavailable players before generating predictions and squad recommendations.

Finally, the application could be improved to support user accounts, allowing managers to save their squad, track predictions over time and compare their team's performance against the model's recommendations across the season.

6.5 Summary

This project set out to build a data-driven tool to help FPL managers make better decisions, and that goal was achieved. The FPL Intelligence Platform is a fully deployed, publicly accessible application that automatically collects data, predicts player points using a machine learning model and generates optimal squads using Linear Programming, all without any manual intervention once deployed.

Beyond the technical outcomes, this project provided valuable experience in building and deploying a complete software system from research through to deployment. The challenges along the way, debugging prediction inconsistencies, restructuring notebook code into a deployed application and fixing feature engineering bugs, were all opportunities to develop a better understanding of the technologies involved and the software development process.

The system is live, publicly accessible and updating automatically throughout the season. It is the kind of tool that would have been useful at the start of the season as an FPL manager myself, and building it from scratch has been the best part of final year.

Appendix A

A.1 Source Code

The full source code for the FPL Intelligence Platform is available on GitHub at:
`https://github.com/JohnsonMununkum/FantasyPremierLeaguewithML`

A.2 Live Application

The deployed web application is accessible at:
`https://fpl-intelligence-web.onrender.com`

A.3 Installation Instructions

To run the application locally:

1. Clone the repository:
   `git clone https://github.com/JohnsonMununkum/FantasyPremierLeaguewithML`
2. Navigate to the project directory: `cd FantasyPremierLeaguewithML/flask_app`
3. Install dependencies: `pip install -r requirements.txt`
4. Run the application: `python app.py`
5. Open a browser and go to `http://localhost:5000`

To populate the database with the latest FPL data, call the update endpoint:
`GET http://localhost:5000/api/trigger-update`

Bibliography

[1] Premier League, "Fantasy premier league," https://fantasy.premierleague.com, 2024, accessed: 2025. [Online]. Available: https://fantasy.premierleague.com

[2] G. B. Dantzig, *Origins of the simplex method*. ACM, 1990. [Online]. Available: https://dl.acm.org/doi/10.1145/87252.88093

[3] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[4] R. Pollard and G. Pollard, "Home advantage in football: A current review of an unsolved puzzle," *The Open Sports Sciences Journal*, vol. 1, no. 1, 2008. [Online]. Available: https://doi.org/10.2174/1875399X00801010012

[5] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015. [Online]. Available: https://www.pearson.com/en-gb/subject-catalog/p/software-engineering/P200000005434

[6] G. Van Rossum and F. L. Drake, "Python 3 reference manual," Scotts Valley, CA, 2009. [Online]. Available: https://docs.python.org/3/reference/

[7] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[8] M. Grinberg, *Flask Web Development*, 2nd ed. O'Reilly Media, 2018. [Online]. Available: https://www.oreilly.com/library/view/flask-web-development/9781491991725/

[9] M. Owens, *The Definitive Guide to SQLite*. Apress, 2006. [Online]. Available: https://doi.org/10.1007/978-1-4302-0172-4

[10] S. Mitchell *et al.*, "Pulp: A linear programming toolkit for python," https://coin-or.github.io/pulp/, 2011. [Online]. Available: https://coin-or.github.io/pulp/

[11] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[12] D. Crockford, "The application/json media type for javascript object notation (json)," https://www.rfc-editor.org/rfc/rfc4627, 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4627

[13] Premier League, "Fantasy premier league api," https://fantasy.premierleague.com/api/bootstrap-static/, 2024, accessed: 2025. [Online]. Available: https://fantasy.premierleague.com/api/bootstrap-static/

[14] W. McKinney, "Data structures for statistical computing in python," in *Proceedings of the 9th Python in Science Conference*, vol. 445, 2010, pp. 51–56. [Online]. Available: https://conference.scipy.org/proceedings/scipy2010/mckinney.html

[15] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997. [Online]. Available: https://www.cs.cmu.edu/~tom/mlbook.html

[16] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. [Online]. Available: https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/

[17] A. Zheng and A. Casari, *Feature Engineering for Machine Learning*. O'Reilly Media, 2018. [Online]. Available: https://www.oreilly.com/library/view/feature-engineering-for/9781491953235/

[18] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785

[19] P. J. Eby, "Pep 3333 – python web server gateway interface v1.0.1," https://peps.python.org/pep-3333/, 2010. [Online]. Available: https://peps.python.org/pep-3333/

[20] M. Otto and J. Thornton, "Bootstrap: The most popular html, css, and js library in the world," https://getbootstrap.com, 2011. [Online]. Available: https://getbootstrap.com

[21] S. Chacon and B. Straub, *Pro Git*. Apress, 2014. [Online]. Available: https://git-scm.com/book/en/v2

[22] GitHub, "Github actions documentation," https://docs.github.com/en/actions, 2023. [Online]. Available: https://docs.github.com/en/actions

[23] Render, "Render documentation," https://render.com/docs, 2023. [Online]. Available: https://render.com/docs

[24] R. Baboota and H. Kaur, "Predictive analysis and modelling football results using machine learning approach for english premier league," *International Journal of Forecasting*, vol. 35, no. 2, pp. 741–755, 2019. [Online]. Available: https://doi.org/10.1016/j.ijforecast.2018.01.001